

The Smart Airport System is a distributed microservice architecture designed to alleviate bottlenecking across checkpoints through multisensor telemetry, Computer Vision (CV), and predictive ML models.

1.1 Project Objectives

- **Surveillance & Tracking:** Ingest high-framerate IP camera feeds and optical LiDAR/IoT passenger counters.

| ID    | Module        | Description                                                               |
|-------|---------------|---------------------------------------------------------------------------|
| FR-01 | Ingestion     | Ingest RTSP video feeds & HTTP telemetry at $\geq 15$ FPS per channel.    |
| FR-02 | Vision Engine | Bounding-box passenger detection & trajectory tracking (YOLOv8/DeepSORT). |
| FR-03 | Analytics     | Compute live queue lengths and wait times updated every 5s.               |

|       |           |                                                                            |
|-------|-----------|----------------------------------------------------------------------------|
| FR-04 | Predictor | Forecast queue congestion for next 30 min and suggest optimal lane counts. |
| FR-05 | Dashboard | Real-time React/WebSocket UI & push alerts for wait times >15 min.         |

# SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

|             |                              |
|-------------|------------------------------|
| Name:       | M.HANIF RIYAZ                |
| Reg No:     | 192521123                    |
| Subject:    | SOFTWARE ENGINEERING (CSA10) |
| Assessment: | CO3 Assessment Tool 1        |
| Date:       | August 16, 2026              |

## ASSIGNED PROBLEM STATEMENT

### Smart Airport Passenger Flow & Queue Management System

Airports handle millions of passengers annually. AI, Computer Vision, IoT sensors, and cloud analytics enable airports to monitor passenger flow in real time and optimize resource allocation.

## 1. Problem Analysis

- **Predictive Queue Analytics:** Employ deep learning (YOLOv8, DeepSORT, LSTM) to anticipate congestion 15–45 min in advance.
- **Resource Optimization:** Dynamic staff allocation algorithms recommending lane status changes.
- **Containerized Architecture:** High availability and zero-downtime deployment using Git and Docker.

1.2 Functional Requirements Matrix

2. Project Structure Design



3. Git Repository Initialization

```
git init smart-airport-queue-management
cd smart-airport-queue-management
git config user.name "M.HANIF RIYAZ"
git config user.email "hanif.riyaz@simats.edu"
git branch -M main
git remote add origin https://github.com/simats-cse/smart-airport-queue-management.git
```

4. Git Branching Strategy

| Branch      | Purpose & Governance                                                |
|-------------|---------------------------------------------------------------------|
| main        | Production branch. Direct commits blocked. Requires 2 PR approvals. |
| development | Integration branch for continuous testing & staging.                |
| feature/*   | Isolated feature development (e.g., feature/yolodetector).          |
| release/*   | Release candidate testing & final prep.                             |

|          |                               |
|----------|-------------------------------|
| hotfix/* | Emergency production patches. |
|----------|-------------------------------|

## 5. Version Control Workflow

```
# Development feature workflow git checkout
development git pull origin development git
checkout -b feature/cv-stream-ingestion git add
src/vision_processor/stream_ingestor.py
git commit -m "feat(vision): add RTSP stream connector with reconnect
logic" git fetch origin git rebase origin/development
git push -u origin feature/cv-stream-ingestion
```

## 6. Dockerfile Development

```
FROM python:3.10-slim AS builder
WORKDIR /app RUN apt-get update && apt-get install -y --no-install-
recommends gcc build-essential && rm -rf /var/lib/apt/lists/* COPY
requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.10-slim AS runner
WORKDIR /app
RUN groupadd -r airportuser && useradd -r -g airportuser airportuser
COPY --from=builder /root/.local /home/airportuser/.local
COPY . /app
ENV PATH=/home/airportuser/.local/bin:$PATH
RUN chown -R airportuser:airportuser /app
USER airportuser
EXPOSE 8000 HEALTHCHECK --interval=30s --timeout=5s --start-
period=10s --retries=3 CMD curl -f http://localhost:8000/health
|| exit 1 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0",
"--port",
"8000"]
```

---

## 7. Docker Compose Orchestration

```
version: '3.8'
services:
  postgres-db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: airport_db
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - airport_internal_net

  backend-api:
    build: ./src/backend
    ports:
      - "8000:8000"
    networks:
      - airport_internal_net
      - airport_public_net
    depends_on:
      - postgres-db

networks:
  airport_internal_net:
    internal: true
  airport_public_net:
    driver: bridge

volumes:
  postgres_data:
```

## 8. Benefits & System Improvements

---

- **Portability & Consistency:** Microservices execute identically across edge gateways and cloud nodes.
- **Resource Efficiency:** Multi-stage Docker builds reduce image sizes from ~1.2GB down to 180MB.
- **Future Expansion:** Planned migration to Kubernetes (k8s) for dynamic pod auto-scaling during high-traffic holiday seasons.